

Laboratorio di Programmazione

Lezione 4: Iterazione

1	Qual è l'output?	2
2	Qual è l'output?	3
3	Trova gli errori.....	4
4	Tabellina	5
5	Somma di numeri in intervallo.....	6
6	Operazioni con un numero variabile di valori	7
7	Media	8
8	Indovina il numero	9
9	Fizz Buzz.....	10
10	Divisori	11
11	Operazioni	12
12	Radice approssimata	13
13	Sequenza pazzarella.....	14

1 Qual è l'output?

Inserendo da standard input 10, qual è l'output di questo programma? E inserendo 11? Che cosa fa questo programma?

```
1 package main
2 import "fmt"
3
4 func main() {
5     var n int
6     fmt.Print("Inserisci un numero: ")
7     fmt.Scan(&n)
8
9     var i int;
10    for i = 0; i <= n; i += 2 {
11        fmt.Print(i, " ")
12    }
13    fmt.Println()
14 }
```

2 Qual è l'output?

Inserendo da standard input 36, qual è l'output di questo programma? E inserendo 32? Che cosa fa questo programma? Quali sono le differenze con il programma dell'esercizio precedente?

```
1 package main
2 import "fmt"
3
4 func main() {
5
6     var n int
7
8     fmt.Print("Inserisci un numero: ")
9     fmt.Scan(&n)
10
11     var i int;
12     for i = 1; i <= n; i *= 2 {
13         fmt.Print(i, " ")
14     }
15
16     fmt.Println()
17
18 }
```

3 Trova gli errori

Questo programma è errato e non produce l'output descritto nel commento. Correggilo (3 errori: 1 di sintassi, 2 di logica del programma).

```
1  /*
2  Il programma legge da standard input un intero n,
3  e stampa su una singola riga tutti gli interi da 1 a n,
4  separati da spazi.
5  */
6  package main
7
8  import (
9      "fmt"
10 )
11
12 func main() {
13
14     var n int
15
16     fmt.Print("Inserisci un numero: ")
17     fmt.Scan(&n)
18
19     var i int;
20     for i = 1; i < n {
21         fmt.Print(i)
22         i++
23     }
24
25     fmt.Println()
26
27 }
```

4 Tabellina

Scrivere un programma che legge da standard input un numero intero n e stampa la corrispondente tabellina (moltiplicando n per gli interi da 1 a 10).

Esempio d'esecuzione:

```
$ go run tabelline.go
Inserisci un numero: 9
1 x 9 = 9
2 x 9 = 18
3 x 9 = 27
4 x 9 = 36
5 x 9 = 45
6 x 9 = 54
7 x 9 = 63
8 x 9 = 72
9 x 9 = 81
10 x 9 = 90
```

5 Somma di numeri in intervallo

Scrivere un programma che legga da standard input due interi a e b , e stampi a video la somma degli interi *pari* compresi tra a e b (estremi esclusi).

Esempio d'esecuzione:

```
$ go run sommaintervallo.go  
1 9  
Somma = 20
```

```
$ go run sommaintervallo.go  
-3 3  
Somma = 0
```

6 Operazioni con un numero variabile di valori

Scrivere un programma che, dopo aver letto da standard input un intero n , chiede all'utente di inserire n altri interi (sempre da standard input), e infine stampa:

- la somma degli n numeri letti;
- il loro minimo;
- il loro massimo;
- il numero di interi letti che sono strettamente positivi, che sono strettamente negativi, e che sono uguali a zero.

Esempio d'esecuzione:

```
$ go run nnumer.go
9
Inserisci 9 numeri:
1 -2 3 -4 5 -6 7 -8 9
somma: 5
valore minimo: -8
valore massimo: 9
interi > 0: 5
interi < 0: 4
interi = 0: 0
```

7 Media

Scrivere un programma che legga da standard input una sequenza di numeri reali strettamente positivi. La lettura termina quando viene letto un numero minore o uguale a 0. Il programma deve stampare a video il risultato della media aritmetica dei valori inseriti.

Esempio d'esecuzione:

```
$ go run medie.go
```

```
Inserisci una sequenza di numeri (interrompi con numero<=0): 4 6 8 0
```

```
Media aritmetica: 6
```

```
$ go run medie.go
```

```
Inserisci una sequenza di numeri (interrompi con numero<=0): 3 5 2 6 1 -1
```

```
Media aritmetica: 3.4
```

8 Indovina il numero

Scrivere un programma che genera un intero casuale n nell'intervallo $[1, 100]$. Il programma chiede quindi di inserire da standard input un intero k . Se $k = n$, allora il programma termina congratulandosi; altrimenti rivela se k è maggiore o minore di n , e chiede nuovamente.

Suggerimento: per generare un numero intero casuale utilizzate le funzioni dei package `math/rand` e `time`, come segue:

```
1 /* inizializzazione del generatore di numeri casuali */
2 rand.Seed(int64(time.Now().Nanosecond()))
3
4 /* generazione di un numero casuale in [0, 99] */
5 var numeroGenerato int = rand.Intn(100)
```

Esempio d'esecuzione:

```
$ go run indovina.go
Tentativo n° 1: 50
Tropo basso! Riprova!
Tentativo n° 2: 75
Tropo alto! Riprova!
Tentativo n° 3: 63
Tropo basso! Riprova!
Tentativo n° 4: 68
Tropo alto! Riprova!
Tentativo n° 5: 66
Hai indovinato in 5 tentativi!
```

9 Fizz Buzz

Scrivere un programma che legga da standard input un intero n . Il programma deve stampare a video la sequenza di interi $1, \dots, n$ con le seguenti modifiche:

- ogni intero divisibile per 3 è rimpiazzato dalla stringa Fizz
- ogni intero divisibile per 5 è rimpiazzato dalla stringa Buzz
- ogni intero divisibile sia per 3 che per 5 è rimpiazzato dalla stringa FizzBuzz

Esempio d'esecuzione:

```
$ go run fizzbuzz.go
```

```
6
```

```
1 2 Fizz 4 Buzz Fizz
```

```
$ go run fizzbuzz.go
```

```
20
```

```
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz 16 17 Fizz 19 Buzz
```

10 Divisori

Scrivere un programma che legga da standard input un numero intero n e stampi a video i suoi *divisori propri* (cioè quelli minori di n). Ad esempio, i divisori di 6 sono 1, 2, 3, 6, mentre i suoi divisori propri sono 1, 2, 3.

Esempio d'esecuzione:

```
$ go run divisori.go  
Inserisci numero: 6  
Divisori di 6: 1 2 3
```

```
$ go run divisori.go  
Inserisci numero: 10  
Divisori di 10: 1 2 5
```

11 Operazioni

Create un programma che legge da standard input due interi x, y e stampa:

- il maggiore tra x e y
- il minore tra x e y
- la somma di x e y
- la differenza tra il maggiore e il minore
- il rapporto (divisione) tra x e y
- il prodotto tra x e y
- il valore medio di x e y
- se $y \geq 0$, il valore di x^y , usando sia un ciclo for che `math.Pow`

Esempio d'esecuzione:

```
$ go run operazioni.go
Inserisci due numeri interi:
2 4
Maggiore: 4
Minore: 2
Somma: 6
Differenza: 2
Prodotto: 8
Divisione: 0.5
Valore medio: 3
Potenza (ciclo for): 16
Potenza (math.Pow): 16
```

12 Radice approssimata

Scrivere un programma che legga da standard input due numeri in virgola mobile, x e ϵ , e trovi la migliore approssimazione a $\sqrt{x}$ che ha la forma $n \cdot \epsilon$, dove n è un numero naturale. In altre parole cerca la migliore approssimazione a $\sqrt{x}$ tra i valori $0, \epsilon, 2\epsilon, 3\epsilon, \dots$. Gestire opportunamente gli errori, inclusi l'inserimento di valori errati per x o ϵ .

Esempio d'esecuzione:

```
$ go run radice.go
Numero del quale vuoi approssimare la radice (x): 10
Precisione (epsilon): 0.1
3.2 (errore: 10.24 - 10 = 0.24)

$ go run radice.go
Numero del quale vuoi approssimare la radice (x): 14.5
Precisione (epsilon): 0.01
3.81 (errore: 14.5161 - 14.5 = 0.0161)
```

13 Sequenza pazzarella

Scrivi un programma in Go che legge un intero $n \geq 1$ da standard input, e poi stampa esattamente n numeri, partendo da 1, secondo le seguenti regole:

1. Non stampa mai i multipli di 3.
2. Se stampa un numero divisibile per 7, il programma non stampa il numero successivo (es, se stampa 14, allora non stampa 15).
3. Se stampa un numero la cui ultima cifra è 6, il programma non stampa i due numeri successivi (es, se stampa 16, allora non stampa né 17 né 18). *Questo ha la precedenza sul caso precedente (se stampa 56, il programma non stampa 57 e 58).*

Esempio d'esecuzione:

```
$ go run pazzarella.go
```

```
Inserire n: 20
```

```
1
2
4
5
7
10
11
13
14
16
19
20
22
23
25
26
29
31
32
34
```